Indução e Recursão

Definições recursivas de funções e sequências e O princípio da indução.

Waldemar Ferreira Neto
Matemática Discreta 1

Introdução

- Muitas das funções matemáticas podem ser implementadas em algoritmos
- Considere o Fatorial

$$fat(n) = \begin{cases} 1, n = 0 \\ n \times (n-1), n > 0 \end{cases}$$

Formas de Fatorial

Chama a si mesmo em cada iteração

- Função fatorial:

$$
\begin{aligned}
n! &= n \times (n-1) \times (n-2) \times (n-3) \times \cdots \times 3 \times 2 \times 1 \\
&= n \times \boxed{(n-1)!}
\end{aligned}
$$

- Outra forma de definição seria:

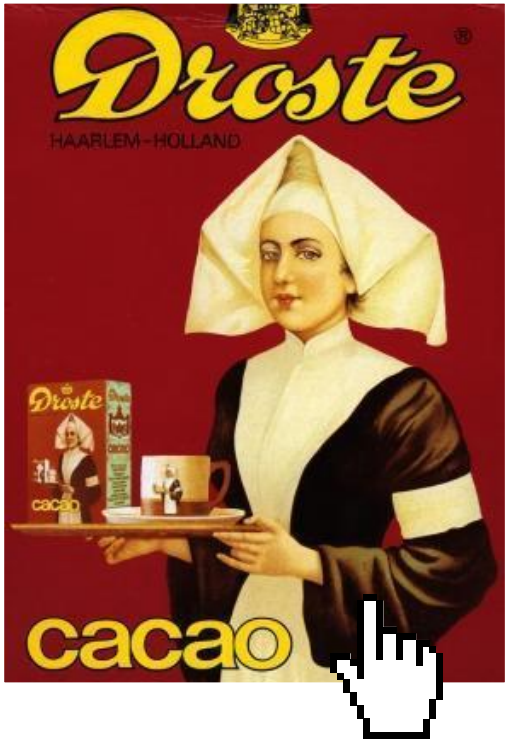
$$
fat(n) = \begin{cases} 1, n = 0 \\ n \times \boxed{(n-1)}, n > 0 \end{cases}
$$

Iterações

Há algo de peculiar nessa imagem?

Iterações

E qual a semelhança com a seguinte imagem?



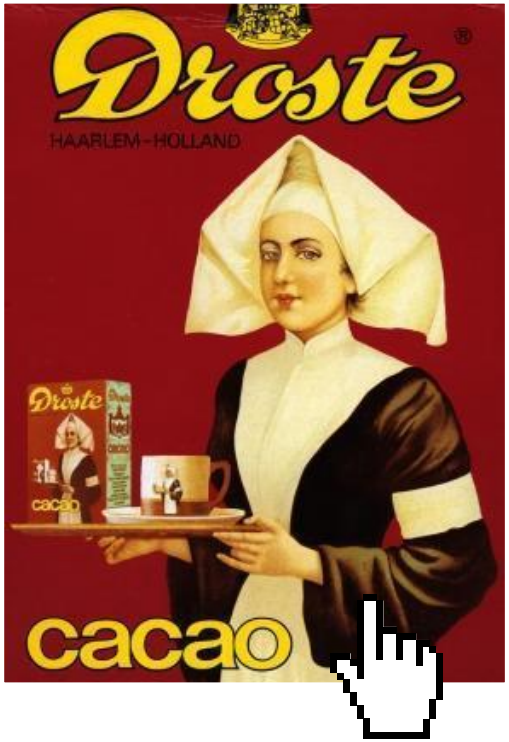
Iterações

Uma imagem que faz referência a si mesma

- **Efeito Droste**

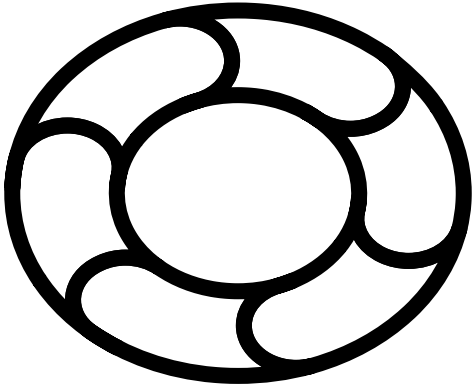
Será que podemos usar essa ideia na programação?

- **Recursividade**



?

Recursividade na programação é quando uma função invoca ela mesma



“To understand recursion, you first have to understand recursion.”

—Stephen Hawking

Exemplo

Escreva uma função que recebe como parâmetro um número inteiro, e retorna o seu **fatorial**!

```
public static int fatorial(int num)
```

Repensar Fatorial

```
4! =1*2*3*4
num! = 1*2*3…(num-1)*num
    4! = 4*3!
    3! = 3*2!
    2! = 2*1!
    1! = 1*0!
    0! = 1
```

```
num! = 1*2*3…(num-1)*num
    num! = num*(num-1)!
    (num-1)! = (num-1)*(num-2)!
    ….
    1! = 1*0!
    0! = 1
```

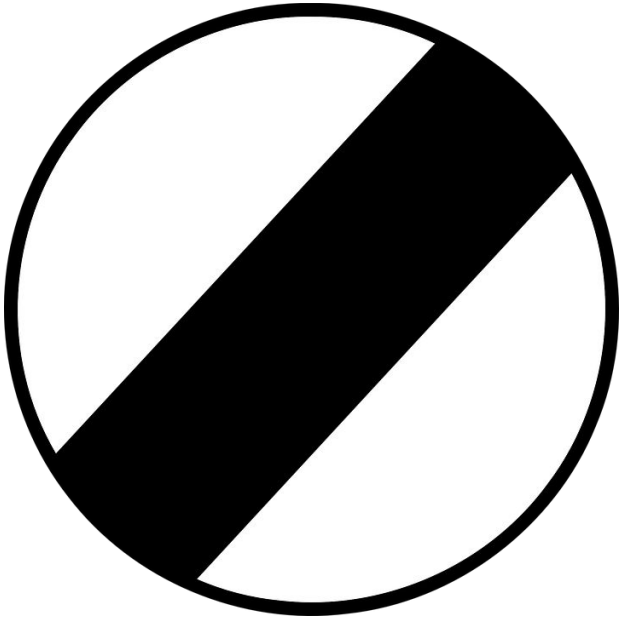
Retomando a definição

Definição:

- É um método de programação no qual uma função pode chamar a si mesma

Em todas as funções recursivas existe:

- **Caso base** (um ou mais) cujo resultado é imediatamente conhecido;
- **Passo recursivo** em que se tenta resolver um subproblema do problema inicial.



Exemplo

- Função fatorial: $$fat(n) = n \times (n-1) \times \ldots \times 1$$

- Outra forma de definição seria:

$$fat(n) = \begin{cases} 1, n = 0 & \text{Caso Base} \\ n \times (n-1), n > 0 & \text{Passo Recursivo} \end{cases}$$

Chamada recursiva

Exemplo

- Função fatorial:

$$fat(n) = \begin{cases} 1, n = 0 \\ n \times (n-1), n > 0 \end{cases}$$

```
int fatorial(int n){
...
}
```

Exemplo

- Função fatorial:

$$fat(n) = \begin{cases} 1, n = 0 \\ n \times (n-1), n > 0 \end{cases}$$

```
int fatorial(int n){
  if(n == 0){
    return 1;
  }
}
```

Caso Base

Exemplo

- Função fatorial:

$$fat(n) = \begin{cases} 1, n = 0 \\ n \times (n-1), n > 0 \end{cases}$$

```
int fatorial(int n){
  if(n == 0){
    return 1;
  } else {
    return n* fatorial(n-1);
  }
}
```

Passo Recursivo

Exemplo

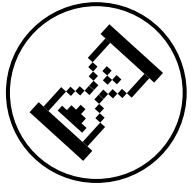
- Função fatorial:

$$fat(n) = \begin{cases} 1, n = 0 \\ n \times (n-1), n > 0 \end{cases}$$

```
int fatorial(int n){
  if(n == 0){
    return 1;
  } else {
    return n* fatorial(n-1);
  }
}
```

Chamada recursiva

Passo Recursivo



Recursividade

E na prática como fica?

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	?
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	?
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	?
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	?
fat(0),n	0
f	?
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	1
fat(0),n	0
f	?
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	1
fat(0),n	0
f	?
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	1
fat(1),n	1
f	?
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	2*1
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	2
fat(2),n	2
f	?
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	3*2
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println
     ("Fatorial de " + n + " = " + r);
}
```

f	6
fat(3),n	3
r	?
n	3

Executando o Método Fatorial

```
static int fatorial(int n){
  int f;
  if(n == 0){
    f = 1;
  } else {
    f = n * fatorial(n-1);
  }
  return f;
}

public static void main(String[] args)
{
  int n = 3, r;
  r = fatorial(n);
  System.out.println("Fatorial de " + n + " = " + r);
}
```

r	6
n	3

Recursividade

- Método Recursivo:
  - Pode ter um algoritmo mais simples
  - Muito uso da memória
- Método Não-Recursivo ou Iterativo:
  - O algoritmo pode ser mais complexo
  - Menor uso de memória

Função Fatorial

Mas como saber se uma função
está correta?

Método da Indução

Técnica de demonstração de corretude

Método Da Indução Matemática

- Agora suponha que sejam verdadeiras as seguintes afirmações sobre as suas habilidades de subir escadas:
  - 1. você pode alcançar o primeiro degrau
  - 2. ao chegar a um degrau qualquer, você sempre pode passar ao degrau seguinte (uma implicação).
- Sequência lógica:
  - Pela sentença 1, você tem garantia de chegar ao primeiro degrau
  - pela 2, você garante que chega ao segundo
  - novamente pela 2, você garante que chega ao segundo
  - novamente pela 2, você garante que chega ao terceiro
  - assim por diante

Princípio Da Indução Matemática

- Técnica de prova de teoremas que estabelece que uma propriedade P(n) é V para todo **n** inteiro e positivo.
- A prova por indução matemática consiste de 2 passos:
  - 1. Passo básico: P(1) é V
  - 2. Passo indutivo: para um k genérico fixo é verdadeiro o condicional:

$$\mathbf{P(k) \rightarrow P(k + 1)}$$

- Observações:
  - 1. Assumir que P(k) é V não é o mesmo que assumir o que queremos provar.
  - 2. Esta é uma técnica de raciocínio dedutivo, usada para provar alguma idéia obtida com um raciocínio indutivo

Princípio Da Indução Matemática

- Exemplo 1: Mostre que, se n é um inteiro positivo:

  $$1 + 2 + \cdots + n = n.(n + 1)/2$$

- Solução:
  - Passo básico: P(1) é V, pois:
    $1 = 1.(1 + 1)/2$
  - Passo indutivo: vamos assumir que P(k) vale, de modo que:
    $1 + 2 + \cdots + k = k.(k + 1)/2$
  - Com base nisto, queremos mostrar que vale:
    $1 + 2 + \cdots + k + (k + 1) = (k + 1).[(k + 1) + 1]/2$ (??)
  - Ora, adicionando-se (k + 1) a ambos os lados de P(k):
    $1 + 2 + \cdots + k + (k + 1) = k.(k + 1)/2 + (k + 1)$
    $= (k + 1).(k + 2)/2$

Princípio Da Indução Matemática

- Exemplo 2: Use a indução matemática para provar que a soma dos primeiros inteiros positivos ímpares é n^2.

  Solução:

- Seja P(n) : “A soma dos primeiros ímpares é n^2”
  - ou: “$1 + 3 + 5 + \cdot \cdot \cdot + (2n - 1) = n^2$”
- Passo básico: comprovar P(1)
  - P(1) estabelece que $1 = 1^2$, o que é V
- Passo indutivo: mostrar que P(k) → P(k + 1) é V
  - Suponha que P(k) é V para um k fixo, ou seja:

    $$1 + 3 + 5 + \cdot \cdot \cdot + (2k - 1) = k^2$$
  - A partir disto, queremos provar que P(k + 1) é V, ou seja:

    $$1 + 3 + 5 + \cdot \cdot \cdot + (2k - 1) + [2(k + 1) - 1] = (k + 1)^2$$

Princípio Da Indução Matemática

- Exemplo 2: Use a indução matemática para provar que a soma dos primeiros inteiros positivos ímpares é n^2.

  Solução:

- Passo indutivo: mostrar que é V a proposição:

  $$1 + 3 + 5 + \cdot \cdot \cdot + (2k - 1) + [2(k + 1) - 1] = (k + 1)^2$$

  - Uma vez que P(k) é V, o lado esquerdo acima fica:

    $$\begin{aligned} k^2 + [2(k + 1) - 1] &= k^2 + (2k + 2 - 1) \\ &= k^2 + 2k + 1 \\ &= (k + 1)^2 \end{aligned}$$

  - Isto mostra que, efetivamente, P(k + 1) segue de P(k).
- Assim, uma vez que P(1) e P(k) → P(k + 1) são V, independente da escolha de k, concluímos que é V a proposição:

  $$\forall n\ P(n)$$

Princípio Da Indução Matemática

- Exemplo 3: Prove que, para qualquer inteiro positivo n, $2^n > n$

  Solução:

- Passo básico: comprovar P(1)
  - P(1) estabelece que $2^1 > 1$, o que é V
- Passo indutivo: mostrar que P(k) → P(k + 1) é V
  - Suponha que P(k) é V para um k fixo, ou seja:

    $2^k > k$
  - Multiplicando os dois lados por 2, temos:

    $2.2^k > 2.k$

    $2^{k+1} > k + k \geq k + 1$

    $2^{k+1} > k + 1$
  - e P(k + 1) é V

Princípio Da Indução Matemática

- Exemplo 4: Prove que $n^2 > 3.n$, para $n \geq 4$.

  Solução:

- Passo básico: neste caso,o passo inicial é P(4):
  - $4^2 > 3.4$, é, efetivamente, V
- Passo indutivo:
  - Hipótese de indução: $k^2 > 3.k$, para $k \geq 4$
  - Queremos mostrar que $(k + 1)^2 > 3.(k + 1)$

$$
\begin{aligned}
(k + 1)^2 &= k^2 + 2.k + 1 \\
&> 3.k + 2.k + 1 && \text{(pela hipótese de indução)} \\
&\geq 3.k + 8 + 1 && \text{(já que } k \geq 4\text{)} \\
&> 3.k + 3 = 3(k + 1) && \text{(já que } k \geq 4\text{)}
\end{aligned}
$$

- Isto mostra que P(k + 1) é V sempre que P(k) é V.

Princípio Da Indução Matemática

- Exemplo 6: Mostre que todo conjunto finito não-vazio é contável, ou seja, pode ser arranjado em uma lista.

  Solução:

- Seja P(n): “se A é qualquer conjunto com $|A| = n \geq 1$, então A é contável.”
- Passo básico:
  - Seja $A = \{x\}$ um conjunto com um elemento.
  - x forma uma sequência cujo conjunto correspondente é A.
  - Então P(1) é V.
- Passo indutivo: usar P(k) para provar P(k + 1) (⇒)

Princípio Da Indução Matemática

- Exemplo 6 (cont.): “Todo conjunto finito não-vazio é contável.”

  Solução:

- Passo indutivo:
  - P(k) é “se A é qualquer conjunto com k elementos, então A é contável”
  - Agora escolha qualquer conjunto B com k + 1 elementos.
  - Escolha um elemento qualquer x em B:
    - B - {x} é um conjunto com k elementos
    - P(k) garante que existe uma sequência x1, x2, . . . , xk que tem B - {x} por seu conjunto correspondente
    - ora, x1, x2, . . . , xk, x tem B por seu conjunto correspondente
    - então B é contável
  - Já que B pode ser qualquer conjunto com k + 1 elementos, P(k + 1) é V se P(k) é V.
  - Conclusão: P(n) é verdadeiro para todo n ≥ 1.

Cuidado!

- Observação: Ao utilizar a indução para provar resultados, tome cuidado para não assumir que “P(k) é V” para forçar o resultado esperado.
- Esta aplicação incorreta do princípio da indução matemática é um erro bastante comum.

POR QUE A INDUÇÃO É VÁLIDA?

Usando Indução

- Por que o método da indução matemática é uma técnica de prova válida?
- Em consequência do “Axioma do bom ordenamento” para os inteiros positivos:
- “Todo sub-conjunto não-vazio do conjunto dos inteiros positivos tem um elemento mínimo.”

Usando Indução

- Axioma do bom ordenamento: “Todo sub-conjunto não-vazio do conjunto dos inteiros positivos tem um elemento mínimo.”
- Argumento:
  - Suponha que sabemos que P(1) é V e que a proposição P(k) → P(k + 1) é V, independente do k escolhido.
  - Agora assuma que existe pelo menos um inteiro positivo para o qual P(n) é F.
  - Então o conjunto S dos inteiros positivos para os quais P(n) é F” é não-vazio.

Usando Indução

- Argumento:
- O conjunto S dos “inteiros positivos para os quais P(n) é F” é não-vazio.
- Logo, pelo bom ordenamento, S tem um elemento mínimo (m):
  - sabemos que m_6= 1, pois assumimos que P(1) é V
  - uma vez que m é positivo e > 1, temos que:

    m - 1 é um inteiro positivo
  - mas m - 1 não pode estar em S, já que m - 1 < m
  - então P(m - 1) deve ser V
- Daí, uma vez que P(k) → P(k + 1) também é V, devemos ter:

  P(m) é V (contradição!)
- Portanto, P(n) deve ser V para todo inteiro positivo n.

ERROS EM PROVAS USANDO INDUÇÃO

Erros

- Em toda prova usando indução matemática, devemos executar de forma correta e completa tanto o passo básico como o passo indutivo.
- Devemos ter cuidado porque algumas vezes é difícil localizar o erro em uma prova por indução defeituosa.

Erros

- Exemplo (1/4): Encontre o erro na falsa prova abaixo de que todo conjunto de linhas no plano não-paralelas aos pares se encontra em um ponto comum.
- “Prova”:
  - Seja P(n): “Todo conjunto de n linhas não-paralelas aos paresno plano se encontra em um ponto comum”.
  - Vamos “provar” que P(n) é V para todo inteiro positivo n ≥ 2.
    - Passo básico: P(2) é V, pois quaisquer duas linhas não-paralelas no plano se encontram em um ponto comum.
    - Passo indutivo: (⇒)

Erros

- Exemplo (2/4): “Todo conjunto de n linhas no plano não-paralelas aos pares se encontra em um ponto comum.”
- “Prova”:
  - Passo indutivo: (⇒)
    - Hipótese: P(k) é V (“k linhas não-paralelas aos pares no plano se encontram em um ponto”)
    - Agora considere k + 1 linhas distintas no plano:
      - pela hipótese: as primeiras k se encotram em um ponto p1
      - também: as últimas k se encontram em um ponto p2
      - agora se fosse p1 6= p2, todas as linhas que os contêm deveriam ser uma só (2 pontos determinam uma reta)
      - portanto: p1 e p2 são o mesmo ponto
      - e o ponto p1 = p2 está em todas as k + 1 linhas

Erros

- Exemplo (3/4): “Todo conjunto de n linhas no plano não-paralelas aos pares se encontra em um ponto comum.”
- “Prova”:
  - Acabamos de mostrar que P(k + 1) é V, assumindo que P(k) é V:
    - ou seja, mostramos que, se assumirmos que todo conjunto de k (k ≥ 2) linhas distintas não-paralelas se encontram em um ponto, isto valerá também para k + 1 linhas.
    - Completamos o passo básico e o passo indutivo de uma prova por indução que parece correta ...

Erros

- Exemplo (4/4): Mostre que há na prova por indução para: “Todo conjunto de n linhas no plano não-paralelas aos pares se encontra em um ponto comum.”
- “Solução”:
  - Note que o passo indutivo requer que k ≥ 3 (!!)
    - Ocorre que não podemos mostrar que P(2) implica em P(3) !
  - Quando k = 2, o nosso objetivo é mostrar que quaisquer 3 linhas distintas não-paralelas aos pares se encontram em um ponto.
  - As duas primeiras linhas se encontram mesmo em um ponto p1 e as duas últimas em um ponto p2.
  - Mas, neste caso, p1 e p2 não precisam ser o mesmo ponto
    - pois apenas a 2a linha é comum a ambos os conjuntos…

Obrigado!

Alguma questão?
waldemar.ferreira@ufrpe.br

netuh.github.com

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**

Contents of this template

Here’s what you’ll find in this **Slidesgo** template:

1. A slide structure based on a presentation for education, which you can easily adapt to your needs. For more info on how to edit the template, please visit Slidesgo School or read our FAQs.
2. To view this template in PowerPoint with the correct fonts, download and install on your computer the **fonts that we have used**. You can learn how to download and install fonts **here.**
3. An assortment of graphic resources that are suitable for use in the presentation can be found in the **alternative resources slides**.
4. A **thanks slide**, which you must keep so that proper credits for our design are given.
5. A **resources slide**, where you’ll find links to all the elements used in the template.
6. **Instructions for use**.
7. Final slides with:
   - The **fonts and colors** used in the template.
   - A **selection of illustrations**. You can also customize and animate them as you wish with the online editor. Visit Storyset to find more.
   - More **infographic resources**, whose size and color can be edited.
   - Sets of **customizable icons** of the following themes: general, business, avatar, creative process, education, help & support, medical, nature, performing arts, SEO & marketing, and teamwork.

You can delete this slide when you’re done editing the presentation.

01 Computer graphics

You can describe the topic of the section here

02 Software and media apps

You can describe the topic of the section here

03 Data modeling/warehousing

You can describe the topic of the section here

04 Modeling, virtual environments

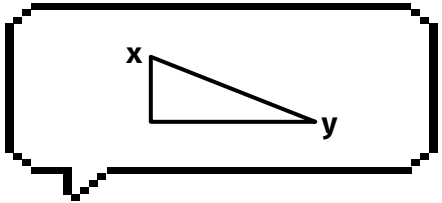
You can describe the topic of the section here

05 Digital & inf. resources design

You can describe the topic of the section here

Introduction

Mercury is the closest planet to the Sun and the smallest one in the Solar System—it’s only a bit larger than the Moon



01

Computer graphics

You can enter a subtitle here if you need it

“This is a quote, words full of wisdom that someone important said and can make the reader get inspired.”

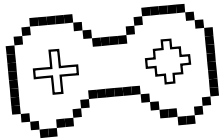
—Someone Famous

The slide title goes here!

Do you know what helps you make your point clear? Lists like this one:

- They’re simple
- You can organize your ideas clearly
- You’ll never forget to buy milk!

And the most important thing: the audience won’t miss the point of your presentation



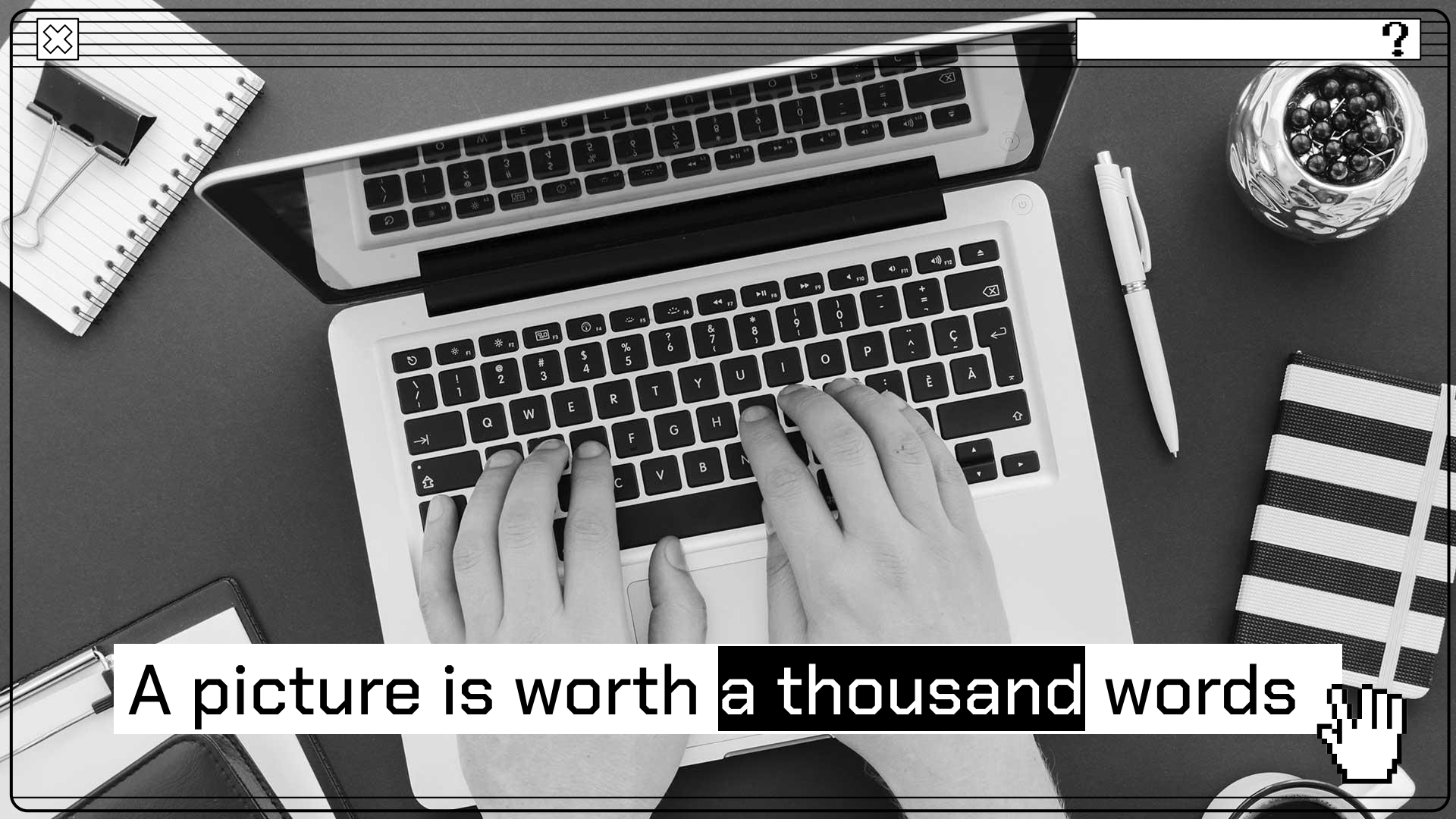
Explore computer science

Theory

Mercury is the closest planet to the Sun and the smallest in the Solar System

Design

Despite being red, Mars is actually a cold place. It’s full of iron oxide dust


?
A picture is worth a thousand words

Whoa!

This can be the part of the presentation where you introduce yourself, write your email…

Computer graphics packages

Mercury

It’s the closest planet to the Sun and the smallest

Venus

Venus is the second planet from the Sun

Mars

Despite being red, Mars is actually a cold place

404 NOT FOUND

02

Computer software and media applications

You can enter a subtitle here if you need it

Job title examples

App developer

Mercury is the closest planet to the Sun and the smallest

Systems engineer

Venus has a beautiful name, but also high temperatures

Software engineer

Earth is the third planet from the Sun and has life

Computer programmer

Despite being red, Mars is actually a cold place

Professional areas

	Freelancer	Employee
Professional, scientific and tech.	✓	
Publishing industries		✓
Electronic product manufacturing	✓	
Insurance and related activities		✓
Management of companies	✓	

What courses to take?

Computational

Mercury is the smallest planet

Computer app

Jupiter is the biggest planet

Development

Despite being red, Mars is a cold place

Programming

Venus has a beautiful name

Mathematics

Neptune is far away from Earth

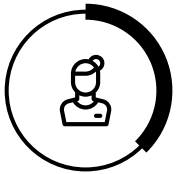
Statistics

Earth is a planet that has life

Labor insertion

Gender

60%



40%

Age

Women ● Men ●

Interests

Follow the link in the graph to modify its data and then paste the new one here. **For more info, click here**

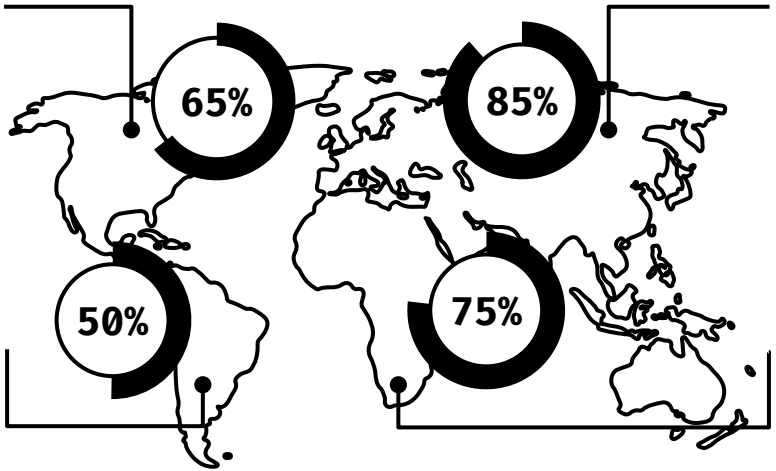
Where are professionals needed?

Venus

Venus has a beautiful name

Mars

Mars is a very cold place



Jupiter

Jupiter is the biggest planet

Saturn

Saturn is the ringed planet

What are other related majors?

Option A

- Saturn is a gas giant and has several rings
- Neptune is the farthest planet from the Sun
- The Sun is the star at the center of the Solar System

Option B

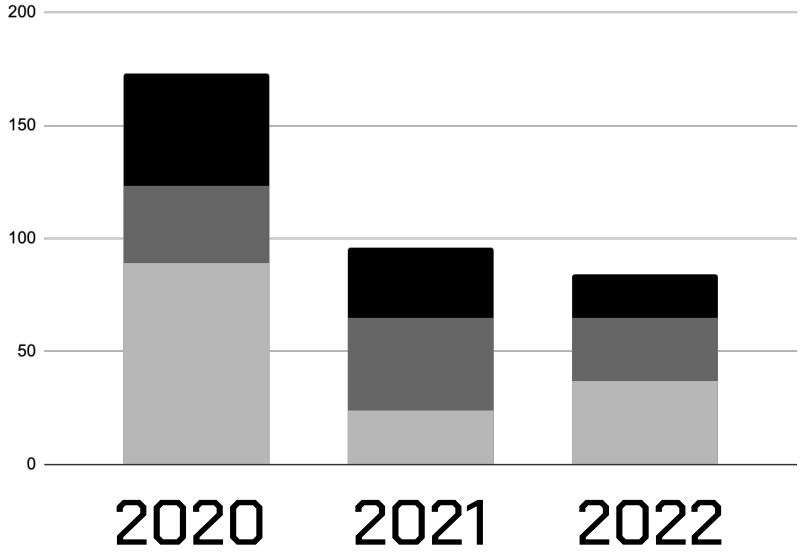
- Venus is the second planet from the Sun
- Despite being red, Mars is a cold place
- Jupiter is the biggest planet in the Solar System

03

Data modeling / warehousing database administration

You can enter a subtitle here if you need it

Graph about database admin.



Follow the link in the graph to modify its data and then paste the new one here. **For more info, click here**

Mercury

It's the closest planet to the Sun

Mars

Mars is actually a very cold place

Neptune

Neptune is far away from Earth

150,000

Big numbers catch your audience’s attention

9h 55m 23s

Jupiter's rotation period

333,000

The Sun’s mass compared to Earth’s

386,000 km

Distance between Earth and the Moon

What courses to take?

Accounting

Mercury is a very small planet

Networks

Venus has a beautiful name

Programming

Mars is actually a very cold place

Logic

Jupiter is the biggest planet

Semantics

Saturn is the ringed planet

Architecture

Neptune is far away from Earth

04

Modeling, virtual environments and simulation

You can enter a subtitle here if you need it

Our major program

Despite being red, Mars is cold

It’s the closest planet to the Sun

Mars

Mercury

2022

2023

2024

2025

Jupiter

It’s the biggest planet of them all

Saturn

It’s composed of hydrogen and helium

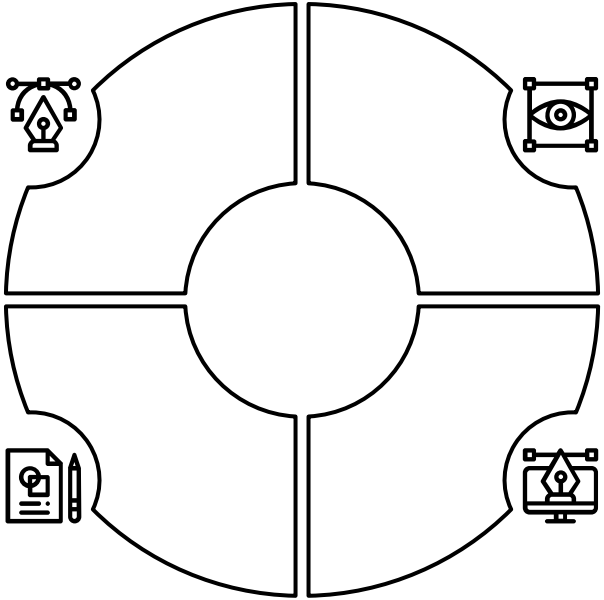
Best modeling courses

Venus

Venus is the second planet from the Sun

Saturn

Saturn is a gas giant and has several rings



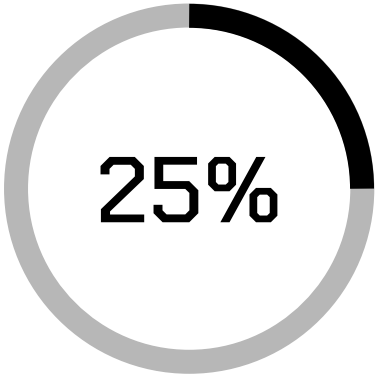
Jupiter

It’s the biggest planet in the Solar System

Mercury

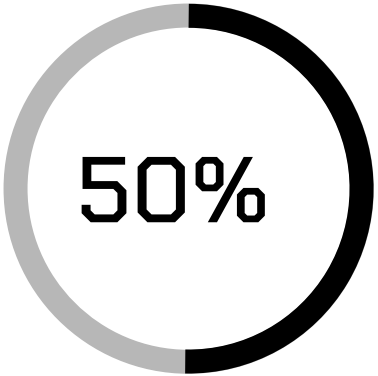
Mercury is the closest planet to the Sun

Employments in this sector



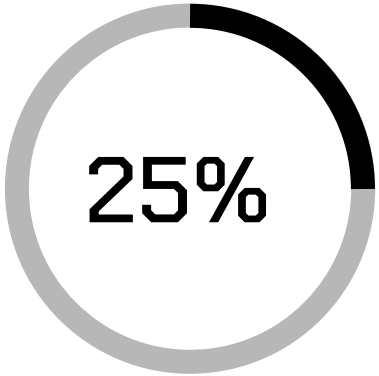
Modeling

Mars is actually a very cold place



Virtual

Neptune is far away from Earth



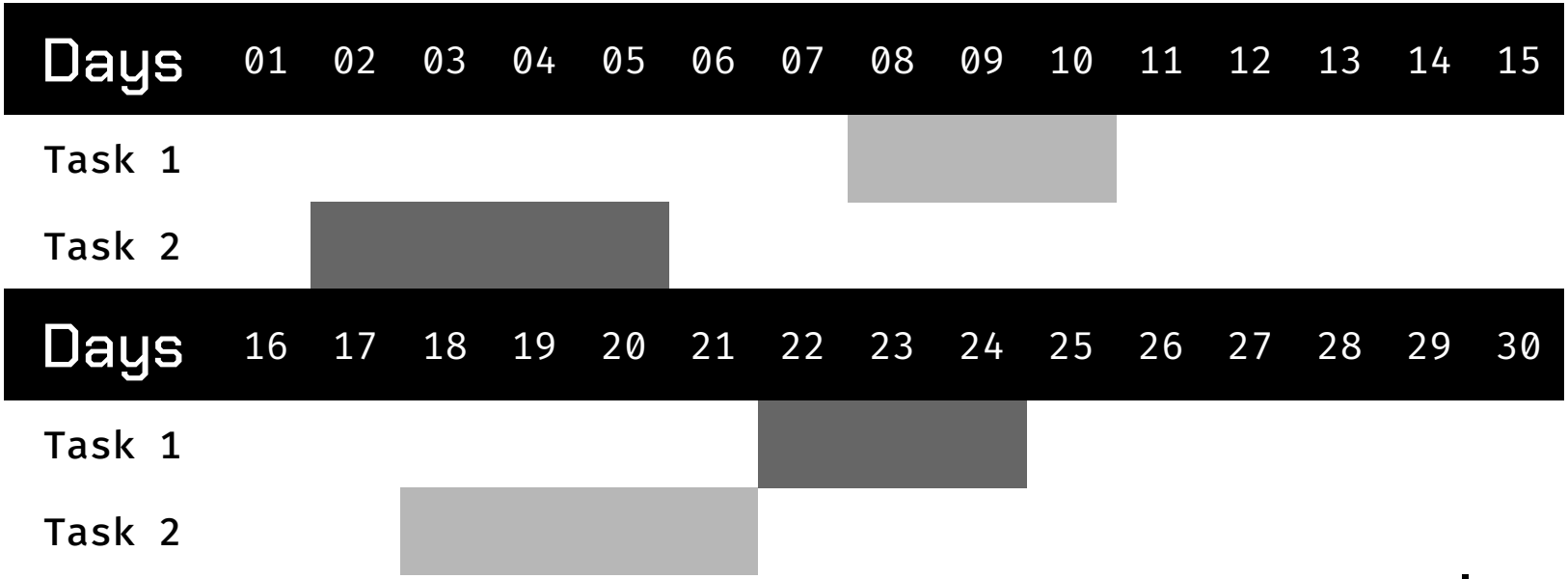
Simulation

Mercury is quite a small planet

Awesome

words

Study schedule



Available

No available

A picture always reinforces the concept

Images reveal large amounts of data, so remember: use an image instead of a long text. Your audience will appreciate it

Our department

Timmy Jimmy

You can replace the image on the screen with your own

Susan Bones

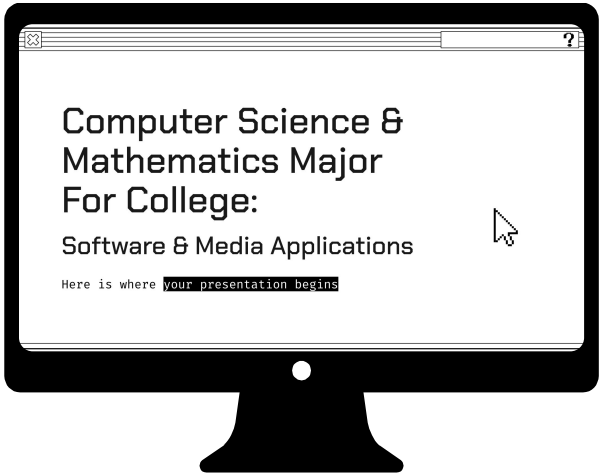
You can replace the image on the screen with your own

John Doe

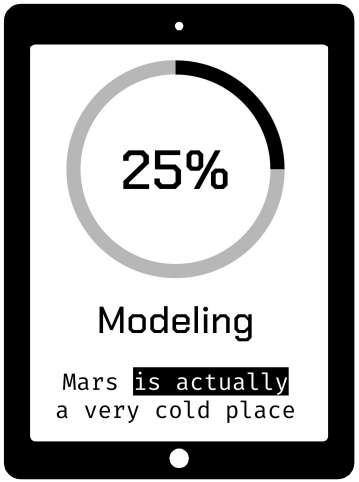
You can replace the image on the screen with your own

Enrollment platform

You can replace the image on the screen with your own work. Just right-click on it and select “Replace image”



Studying app



You can replace the image on the screen with your own work. Just right-click on it and select “Replace image”

Messaging app

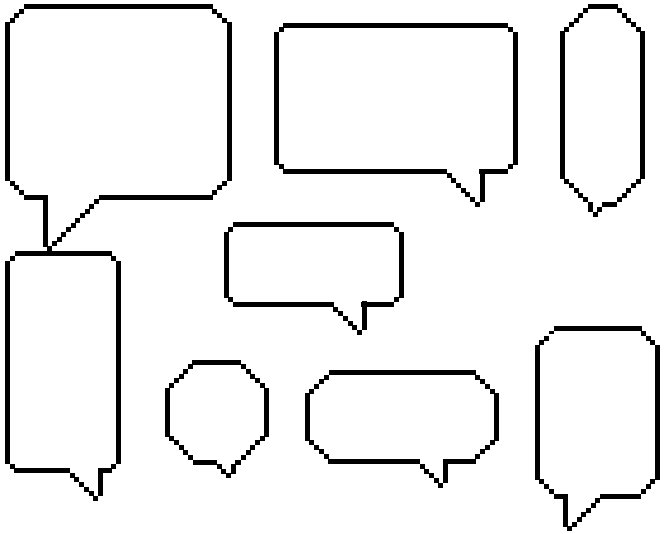
You can replace the image on the screen with your own work. Just right-click on it and select “Replace image”

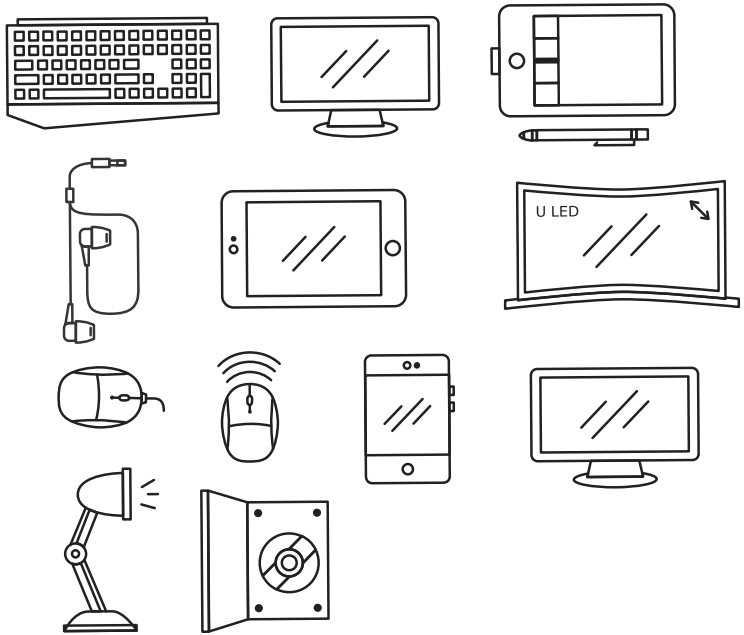
Thanks!

Do you have any questions?
youremail@freepik.com
+91 620 421 838
yourwebsite.com

Please keep this slide for attribution
CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**

Alternative resources





Alternative resources

Did you like the resources on this template? Get them for free at our other websites:

Photos

- Young woman browsing laptop on road trip
- High angle of woman drawing at desk with laptop
- Flat lay desk with laptop and glasses

Vectors

- Black friday background with black and white elements
- Modern cursor set
- Multimedia icons
- Set of white speech bubbles in pixel art style

Resources

Did you like the resources we used in this template? Get them for free at our other websites:

Photos

- Laptop Free
- White keyboard of modern laptop
- Medium shot colleagues back to office

Vectors

- Set of white speech bubbles in pixel art style
- Pixel gaming youtube channel art
- Pixel mouse cursors pointer, arrow and wait
- Vector pixel mouse cursors over white
- Vector pixel mouse cursors over white

Instructions for use

If you have a free account, in order to use this template, you must credit **Slidesgo** by keeping the **Thanks** slide. Please refer to the next slide to read the instructions for premium users.

As a Free user, you are allowed to:

- Modify this template.
- Use it for both personal and commercial projects.

You are not allowed to:

- Sublicense, sell or rent any of Slidesgo Content (or a modified version of Slidesgo Content).
- Distribute Slidesgo Content unless it has been expressly authorized by Slidesgo.
- Include Slidesgo Content in an online or offline database or file.
- Offer Slidesgo templates (or modified versions of Slidesgo templates) for download.
- Acquire the copyright of Slidesgo Content.

For more information about editing slides, please read our FAQs or visit Slidesgo School:

https://slidesgo.com/faqs and https://slidesgo.com/slidesgo-school

Instructions for use (premium users)

As a Premium user, you can use this template without attributing **Slidesgo** or keeping the "**Thanks**" slide.

You are allowed to:

- Modify this template.
- Use it for both personal and commercial purposes.
- Hide or delete the “Thanks” slide and the mention to Slidesgo in the credits.
- Share this template in an editable format with people who are not part of your team.

You are not allowed to:

- Sublicense, sell or rent this Slidesgo Template (or a modified version of this Slidesgo Template).
- Distribute this Slidesgo Template (or a modified version of this Slidesgo Template) or include it in a database or in any other product or service that offers downloadable images, icons or presentations that may be subject to distribution or resale.
- Use any of the elements that are part of this Slidesgo Template in an isolated and separated way from this Template.
- Register any of the elements that are part of this template as a trademark or logo, or register it as a work in an intellectual property registry or similar.

For more information about editing slides, please read our FAQs or visit Slidesgo School:

https://slidesgo.com/faqs and https://slidesgo.com/slidesgo-school

Fonts & colors used

This presentation has been made using the following fonts:

Chakra Petch
(https://fonts.google.com/specimen/Chakra+Petch)

Fira Code
(https://fonts.google.com/specimen/Fira+Code)

#000000	#ffffff	#666666	#b7b7b7

Storyset

Create your Story with our illustrated concepts. Choose the style you like the most, edit its colors, pick the background and layers you want to show and bring them to life with the animator panel! It will boost your presentation. Check out how it works.

Pana

Amico

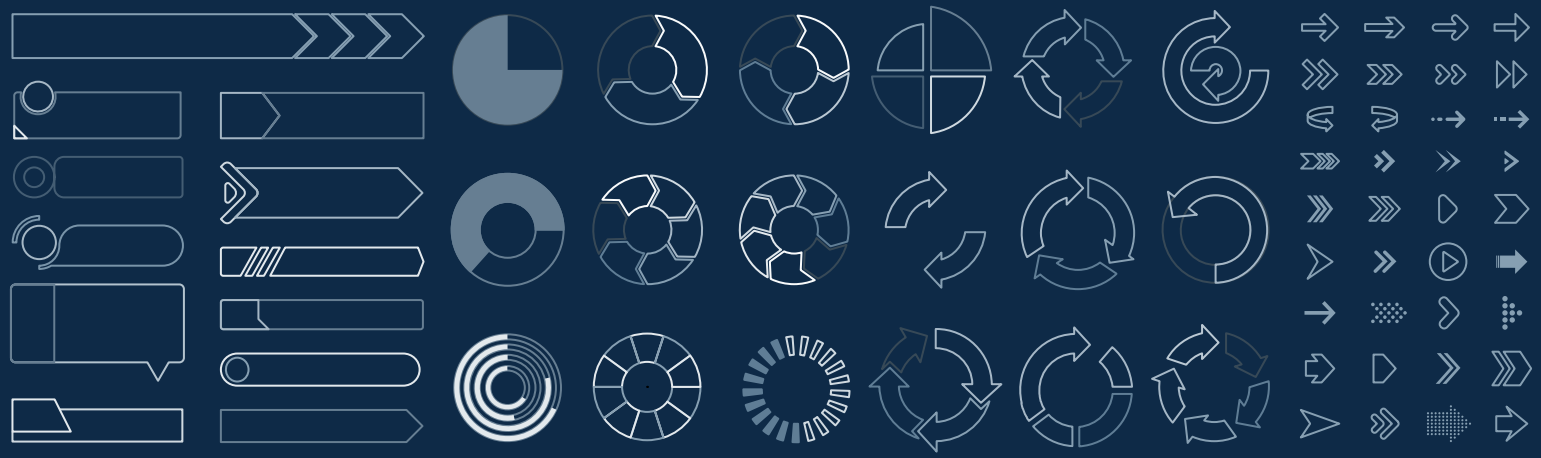
Bro

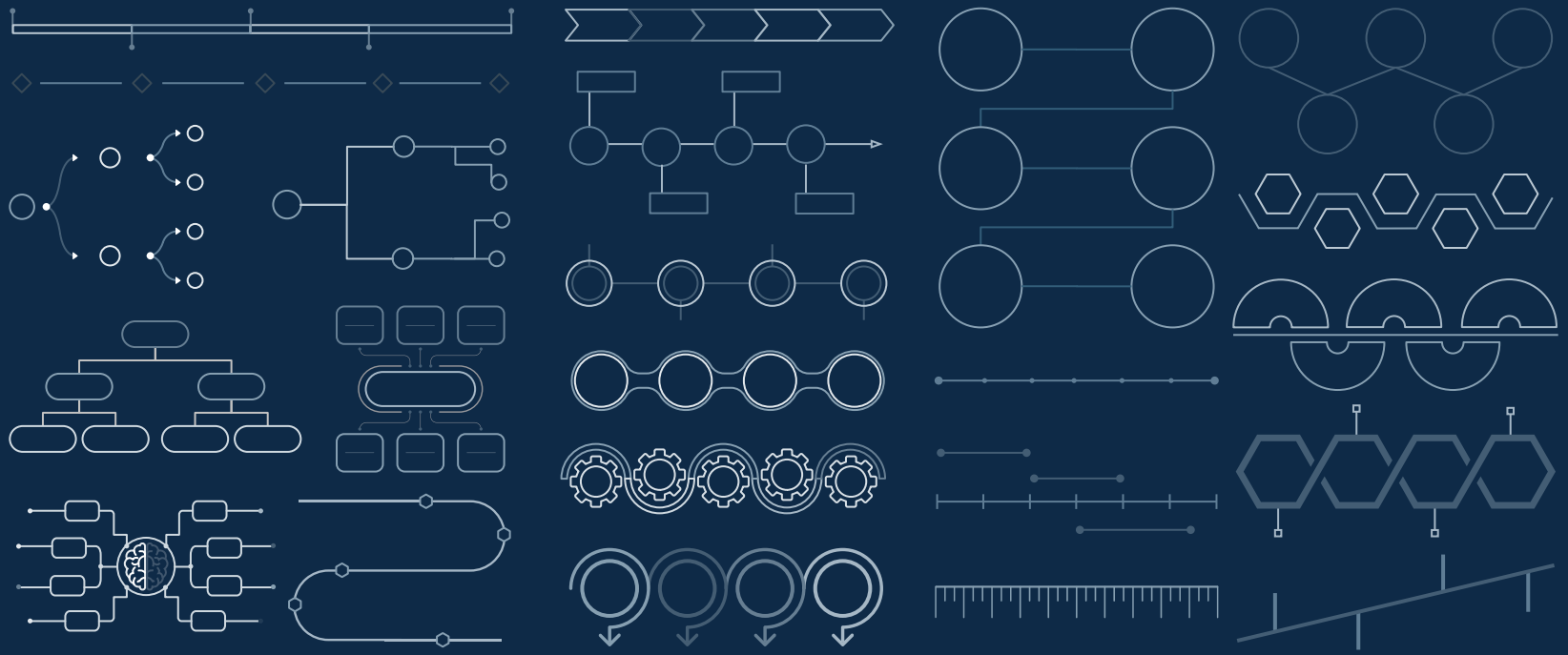
Rafiki

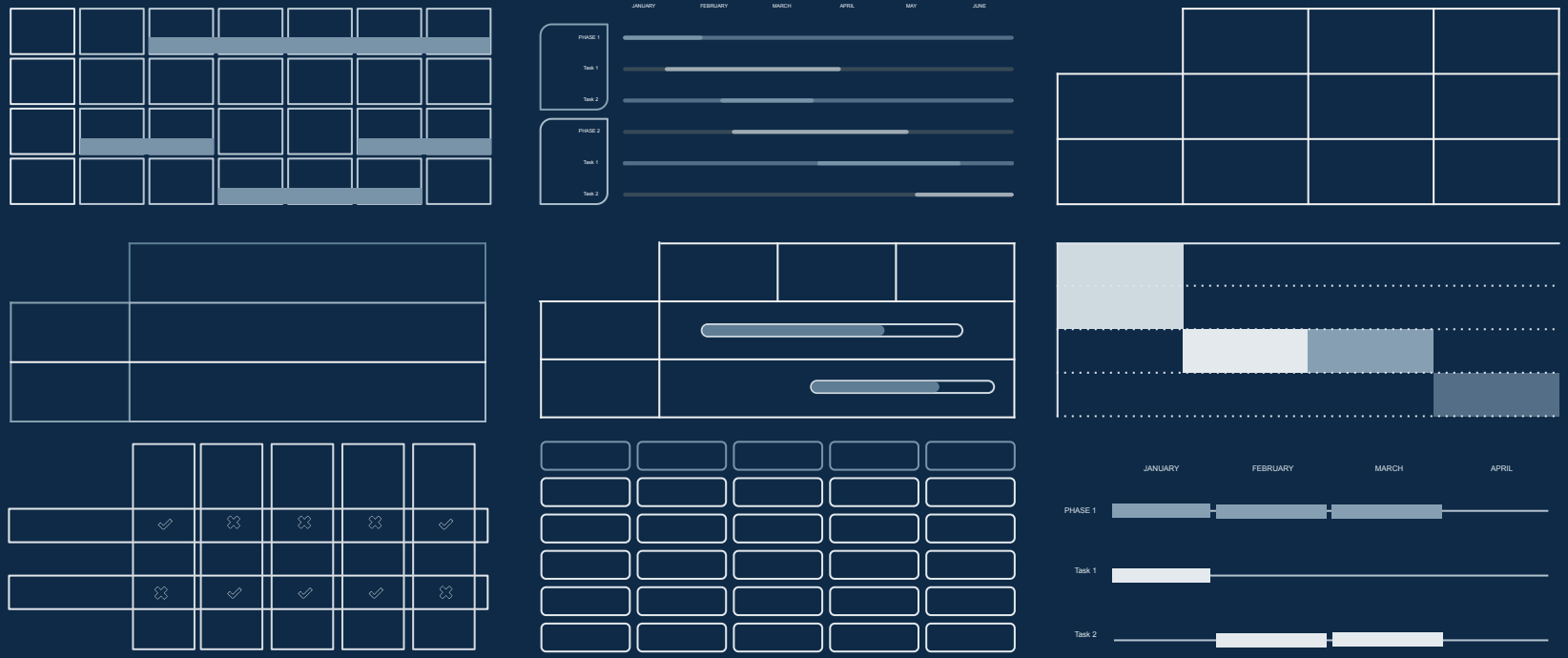
Cuate

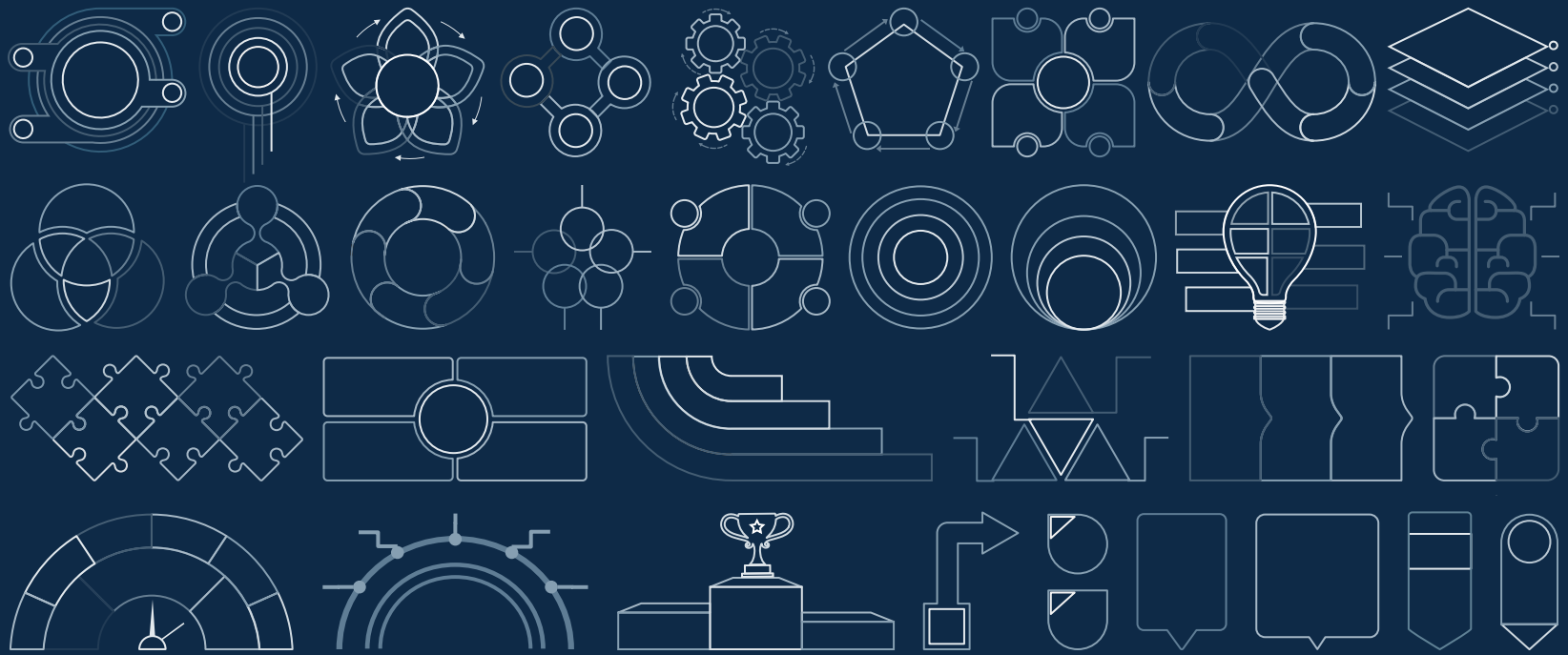
Use our editable graphic resources...

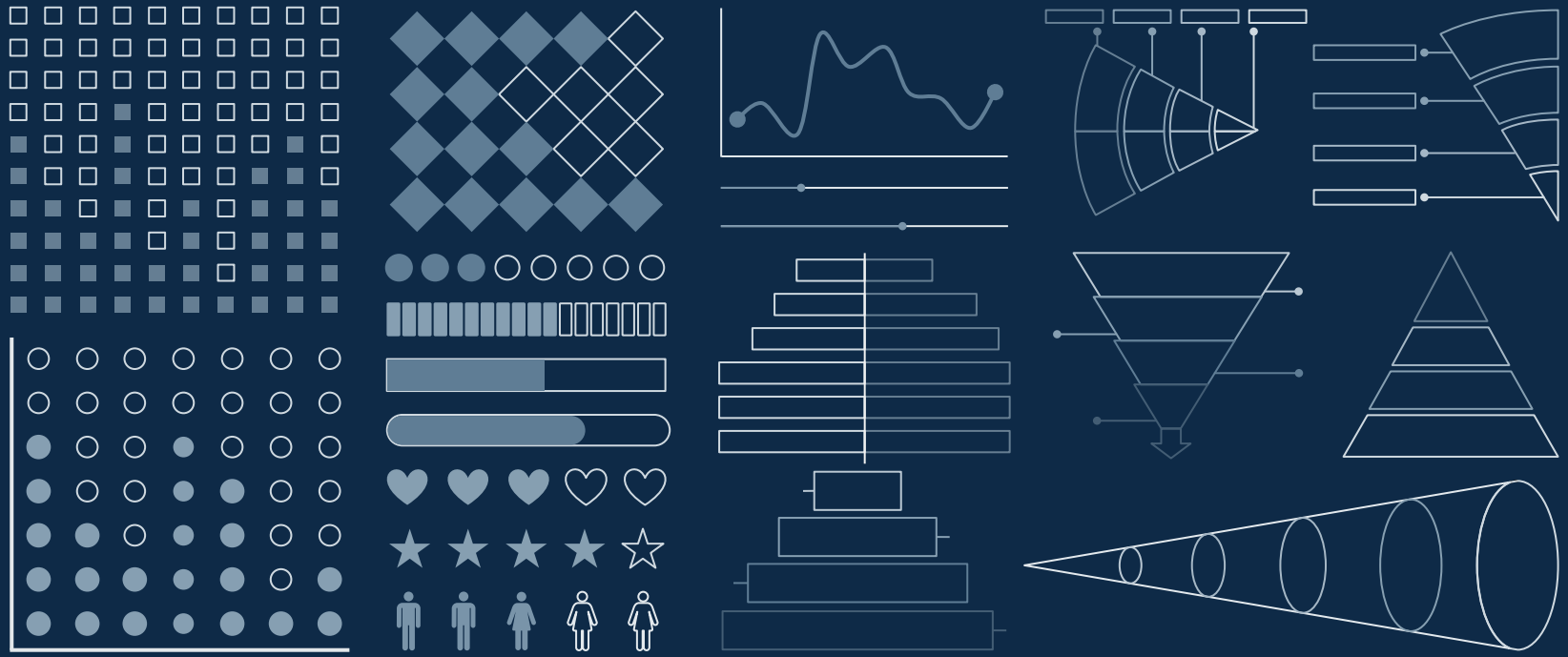
You can easily resize these resources without losing quality. To change the color, just ungroup the resource and click on the object you want to change. Then, click on the paint bucket and select the color you want. Group the resource again when you're done. You can also look for more infographics on Slidesgo.






JANUARY
FEBRUARY
MARCH
APRIL
MAY
JUNE
PHASE 1
Task 1
Task 2
PHASE 2
Task 1
Task 2
JANUARY
FEBRUARY
MARCH
APRIL
PHASE 1
Task 1
Task 2





...and our sets of editable icons

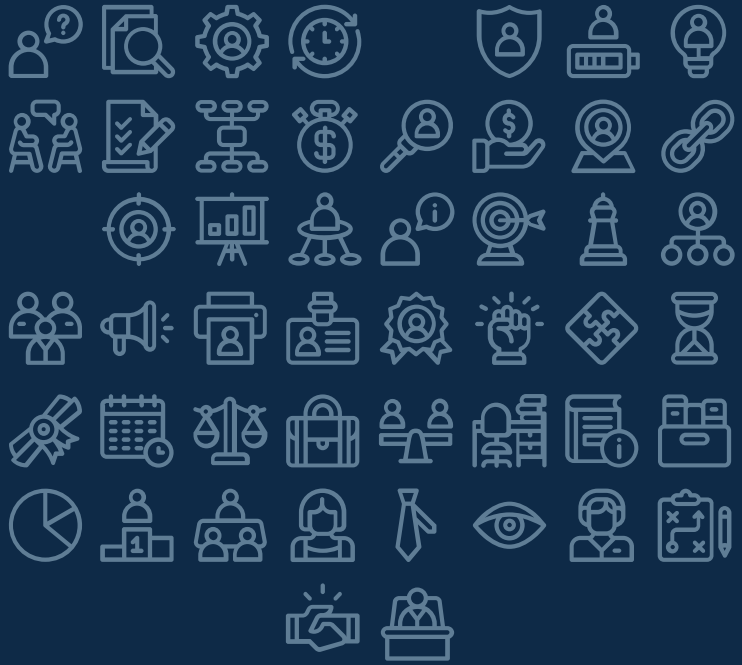
You can resize these icons without losing quality.
You can change the stroke and fill color; just select the icon and click on the paint bucket/pen.
In Google Slides, you can also use Flaticon's extension, allowing you to customize and add even more icons.

Educational Icons

Medical Icons

Business Icons

Teamwork Icons



Help & Support Icons

Avatar Icons

Creative Process Icons

Performing Arts Icons

Nature Icons

SEO & Marketing Icons

slidesgo